PYTHON ACTIVITY-BASED LEARNING

10 Selected Case Studies — Complete Assignment Report

Prepared according to the uploaded Python Activity-Based Learning circular and case-study

list.

Student Name:	Killadi.Hema Durga Venkat_____
Roll Number:	___26000929_____
Branch / Section:	__csm-2_____
Course:	Python Programming — B.Tech 1st Year, 1st Semester
Repository Name:	Python-Activity-Based-Learning

Important: Replace the blank student details before submission. The program rules and rates used below are classroom assumptions for demonstrating Python concepts; use faculty-provided rules if they differ.

Each case study contains a problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, a sample output screenshot, and conclusion.

Selected Case Studies

No.	Case Study	PDF Topic
1	Progressive Electricity Billing	Case Study 2
2	Employee Payroll and Deduction Analysis	Case Study 4
3	Retail Billing with Multiple Discount Rules	Case Study 7
4	Examination Result and Grace-Mark Validation	Case Study 11
5	Attendance Recovery Computation	Case Study 12
6	ATM Transaction Sequence Simulation	Case Study 13
7	Student Timetable Conflict Detection	Case Study 23
8	Password Policy Validation Engine	Case Study 29
9	Student Marks Matrix Analysis	Case Study 42
10	Food Waste Monitoring System	Case Study 53

General execution instructions

Run each program separately in Python IDLE, VS Code, or PyCharm. Enter the sample inputs shown in the test cases. Save a screenshot of the actual program output from your computer if your faculty specifically requires screenshots captured from your own execution environment. The included console-style images are sample output illustrations.

1. Progressive Electricity Billing

Reference: Case Study 2

1. Problem Statement

Develop a Python program to calculate a household electricity bill using progressive unit slabs, a fixed charge, and a surcharge.

2. Objectives

• Read electricity units from the user.

• Calculate energy charges using three consumption slabs.

• Add a fixed charge and a 5% surcharge.

• Display an itemized bill and total amount.

3. Assumptions

• The first 100 units cost Rs. 2 per unit.

• Units from 101 to 200 cost Rs. 3 per unit.

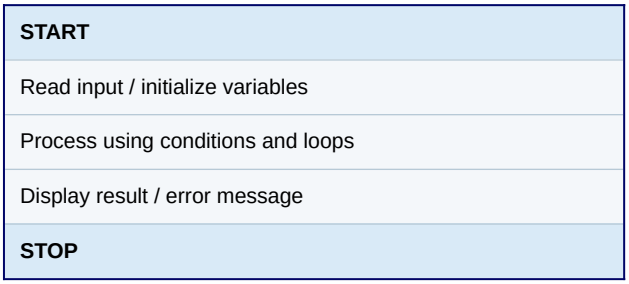
• Units above 200 cost Rs. 5 per unit.

• The fixed charge is Rs. 50 and the surcharge is 5% of the energy charge.

4. Algorithm

1. Start.
2. Read units consumed.
3. If units are negative, display an error.
4. Calculate energy charge using the applicable slabs.
5. Calculate surcharge and total bill.
6. Display the charges.
7. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT units
IF units < 0 THEN
    PRINT invalid input
ELSE
    IF units <= 100 THEN energy = units * 2
    ELSE IF units <= 200 THEN energy = 200 + (units - 100) * 3
ELSE energy = 500 + (units - 200) * 5
    surcharge = energy * 0.05     total = energy + 50 +
surcharge     PRINT itemized bill
END IF
STOP
```

1. Progressive Electricity Billing — Python Source Code

7. Python Source Code

```
units = int(input("Enter electricity units consumed: "))

if units < 0:
```

```
    print("Invalid units")
else:     if units <= 100:
        energy_charge = units * 2
elif units <= 200:
        energy_charge = 100 * 2 + (units - 100) * 3     else:
energy_charge = 100 * 2 + 100 * 3 + (units - 200) * 5

    fixed_charge = 50     surcharge = energy_charge * 0.05
total_bill = energy_charge + fixed_charge + surcharge

    print("Units consumed:", units)
print("Energy charge: Rs.", energy_charge)
print("Fixed charge: Rs.", fixed_charge)
print("Surcharge: Rs.", round(surcharge, 2))
print("Total bill: Rs.", round(total_bill, 2))
```

8. Test Cases

Test Input	Expected Result
150	Energy 350; surcharge 17.50; total Rs. 417.50
50	Energy 100; surcharge 5.00; total Rs. 155.00
-2	Invalid units

9. Sample Output Screenshot

SAMPLE CONSOLE OUTPUT

```
Enter electricity units consumed: 150
Units consumed: 150
Energy charge: Rs. 350
Fixed charge: Rs. 50
Surcharge: Rs. 17.5
Total bill: Rs. 417.5
```

10. Conclusion

The program demonstrates conditional statements and arithmetic operations to calculate an electricity bill using progressive slabs.

2. Employee Payroll and Deduction Analysis

Reference: Case Study 4

1. Problem Statement

Create a program to calculate an employee's gross salary, salary deduction, and net salary from basic pay, allowances, overtime, and incentives.

2. Objectives

• Accept salary components.

• Calculate gross salary.

• Apply a deduction rate based on gross salary.

• Print a simple salary statement.

3. Assumptions

• All salary values must be non-negative.

• Gross salary up to Rs. 20,000 has a 5% deduction.

• Gross salary above Rs. 20,000 and up to Rs. 40,000 has a 10% deduction.

• Gross salary above Rs. 40,000 has a 15% deduction. These are classroom assumptions, not official tax rules.**4.**

Algorithm

1. Start.

2. Read basic salary, allowances, overtime, and incentives.

3. Check that all values are non-negative.

4. Add the values to obtain gross salary.

5. Choose the deduction rate using if-elif-else.

6. Calculate net salary and display the statement.

7. Stop.

5. Flowchart

START
Read input / initialize variables
Process using conditions and loops
Display result / error message
STOP

6. Pseudocode

```
START
INPUT basic, allowances, overtime, incentives
IF any value < 0 THEN PRINT invalid input ELSE
    gross = basic + allowances + overtime + incentives
IF gross <= 20000 THEN rate = 0.05
    ELSE IF gross <= 40000 THEN rate = 0.10
ELSE rate = 0.15     deduction = gross *
rate     net = gross - deduction     PRINT
gross, deduction, net
END IF
STOP
```

2. Employee Payroll and Deduction Analysis — Python Source Code

7. Python Source Code

```
basic = float(input("Enter basic salary: Rs. ")) allowances
= float(input("Enter allowances: Rs. ")) overtime =
float(input("Enter overtime amount: Rs. ")) incentives =
float(input("Enter incentives: Rs. "))

if basic < 0 or allowances < 0 or overtime < 0 or incentives < 0:
    print("Invalid salary details") else:     gross_salary =
basic + allowances + overtime + incentives

    if gross_salary <= 20000:
        deduction = gross_salary * 0.05
elif gross_salary <= 40000:
        deduction = gross_salary * 0.10
else:         deduction = gross_salary *
0.15

    net_salary = gross_salary - deduction
print("Gross salary: Rs.", round(gross_salary, 2))
print("Deduction: Rs.", round(deduction, 2))
print("Net salary: Rs.", round(net_salary, 2))
```

8. Test Cases

Test Input	Expected Result
20000, 3000, 1000, 1000	Gross Rs. 25000; deduction Rs. 2500; net Rs. 22500
10000, 0, 0, 0	Gross Rs. 10000; deduction Rs. 500; net Rs. 9500
-100, 0, 0, 0	Invalid salary details

9. Sample Output Screenshot

SAMPLE CONSOLE OUTPUT

```
Enter basic salary: Rs. 20000
Enter allowances: Rs. 3000
Enter overtime amount: Rs. 1000
Enter incentives: Rs. 1000
Gross salary: Rs. 25000.0
Deduction: Rs. 2500.0
Net salary: Rs. 22500.0
```

10. Conclusion

The program uses variables, arithmetic, and conditional statements to produce a basic salary statement.

3. Retail Billing with Multiple Discount Rules

Reference: Case Study 7

1. Problem Statement

Prepare a retail bill for multiple products, calculate the subtotal, apply a purchase-based discount, add tax, and show the final amount.

2. Objectives

• Accept prices and quantities for multiple products.

• Calculate the subtotal.

• Apply a discount based on subtotal.

• Calculate tax and final bill.

3. Assumptions

• Product prices must be non-negative and quantities must be positive.

• Subtotal of Rs. 5,000 or more receives 10% discount.

• Subtotal from Rs. 2,000 to below Rs. 5,000 receives 5% discount.

• Other purchases receive no discount. Tax is assumed to be 5% after discount.

4. Algorithm

1. Start.
2. Read the number of products.
3. For each product, read price and quantity and validate them.
4. Add each valid product amount to the subtotal.
5. Choose discount based on subtotal.
6. Calculate tax on discounted amount and final bill.
7. Display subtotal, discount, tax, and total.
8. Stop.

5. Flowchart

START
Read input / initialize variables
Process using conditions and loops
Display result / error message
STOP

6. Pseudocode

```
START
INPUT n
SET total = 0
REPEAT n times
    INPUT price, quantity     IF price >=
0 AND quantity > 0 THEN         total =
total + price * quantity     ELSE PRINT
invalid product details
END REPEAT
IF total >= 5000 THEN discount = total * 0.10
ELSE IF total >= 2000 THEN discount = total * 0.05
ELSE discount = 0 tax = (total - discount) * 0.05
final = total - discount + tax PRINT bill details
STOP
```

3. Retail Billing with Multiple Discount Rules — Python Source Code

7. Python Source Code

```
n = int(input("Enter number of products: ")) total
= 0

if n <= 0:
    print("Invalid number of products")
else:     for i in range(n):
        print("Product", i + 1)         price =
float(input("Enter price: Rs. "))         quantity
= int(input("Enter quantity: "))

        if price < 0 or quantity <= 0:
            print("Invalid product details")
else:             total = total + price *
quantity

    print("Subtotal: Rs.", round(total, 2))

    if total >= 5000:
        discount = total * 0.10
elif total >= 2000:
        discount = total * 0.05
else:         discount = 0

    tax = (total - discount) * 0.05
final_bill = total - discount + tax
print("Discount: Rs.", round(discount, 2))
print("Tax: Rs.", round(tax, 2))
    print("Final bill: Rs.", round(final_bill, 2))
```

8. Test Cases

Test Input	Expected Result
one item: price 3000, quantity 1	Subtotal Rs. 3000; discount Rs. 150; tax Rs. 142.50; final Rs. 2992.50
one item: price 1000, quantity 1	Subtotal Rs. 1000; no discount; tax Rs. 50; final Rs. 1050
n = 0	Invalid number of products

9. Sample Output Screenshot

SAMPLE CONSOLE OUTPUT

```
Enter number of products: 1
Product 1
Enter price: Rs. 3000
Enter quantity: 1
Subtotal: Rs. 3000.0
Discount: Rs. 150.0
Tax: Rs. 142.5
Final bill: Rs. 2992.5
```

10. Conclusion

The program practices loops, input validation, arithmetic calculations, and if-elif-else discount rules.

4. Examination Result and Grace-Mark Validation

Reference: Case Study 11

1. Problem Statement

Read marks for three subjects, validate the marks, apply a limited grace-mark rule, and display the student's result and percentage.

2. Objectives

• Store marks in a list.

• Validate that each mark is between 0 and 100.

• Apply up to five grace marks only when needed to reach 40.

• Display final marks, total, percentage, and result.

3. Assumptions

• There are three subjects, each out of 100.

• A mark of 40 is required to pass each subject.

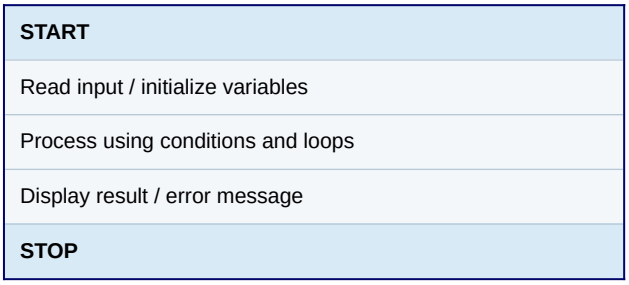
• If a subject mark is 35 to 39, it is raised to 40 for this classroom example.

• This illustrative rule should be replaced if the faculty provides a different policy.

4. Algorithm

1. Start.
2. Read three subject marks into a list.
3. Check whether any mark is outside 0 to 100.
4. If invalid, display an error.
5. Otherwise, apply the grace rule to eligible subjects.
6. Calculate total and percentage.
7. Display PASS if all final marks are at least 40; otherwise display FAIL.
8. Stop.

5. Flowchart



6. Pseudocode

```
START
SET marks = empty list
INPUT marks for 3 subjects
IF any mark < 0 OR mark > 100 THEN PRINT invalid marks
ELSE
    FOR each mark         IF mark < 40 AND 40 - mark <= 5
THEN set mark = 40     total = sum of marks     percentage
= total / 3     IF every mark >= 40 THEN result = PASS
    ELSE result = FAIL
    PRINT marks, total, percentage, result
STOP
```

4. Examination Result and Grace-Mark Validation — Python Source Code

7. Python Source Code

```
marks = []

for i in range(3):
    mark = int(input("Enter marks for subject " + str(i + 1) + ": "))
marks.append(mark)

valid = True for mark in marks:
if mark < 0 or mark > 100:
valid = False

if not valid:
    print("Invalid marks. Enter marks between 0 and 100.") else:
    final_marks = []     for mark in
marks:          if mark < 40 and 40 - mark
<= 5:
            mark = 40
final_marks.append(mark)

    print("Final marks:", final_marks)     total =
sum(final_marks)     percentage = total / 3
print("Total marks:", total)
print("Percentage:", round(percentage, 2), "%")

    if all(mark >= 40 for mark in final_marks):
        print("Result: PASS")
else:         print("Result:
FAIL")
```

8. Test Cases

Test Input	Expected Result
38, 65, 72	Final marks [40, 65, 72]; total 177; percentage 59.0%; PASS
30, 65, 72	Final marks [30, 65, 72]; total 167; percentage 55.67%; FAIL
101, 50, 60	Invalid marks

9. Sample Output Screenshot

SAMPLE CONSOLE OUTPUT

```
Enter marks for subject 1: 38
Enter marks for subject 2: 65
Enter marks for subject 3: 72
Final marks: [40, 65, 72]
Total marks: 177
Percentage: 59.0 %
Result: PASS
```

10. Conclusion

The program demonstrates lists, loops, validation, and conditional result classification.

5. Attendance Recovery Computation

Reference: Case Study 12

1. Problem Statement

Calculate current attendance and find the minimum number of consecutive classes a student must attend to reach a target attendance percentage.

2. Objectives

• Read attended classes and total classes.

• Calculate current attendance percentage.

• Use a loop to find the minimum additional classes needed.

• Display the result.

3. Assumptions

• The target attendance is 75%.

• Every additional class is attended by the student.

• The total number of classes must be positive and attended classes cannot exceed total classes.

4. Algorithm

1. Start.
2. Read attended classes and total classes.
3. Validate the values.
4. Calculate current percentage.
5. Set extra classes to zero.
6. While the projected percentage is below 75%, increase extra by one.
7. Display current attendance and required extra classes.
8. Stop.

5. Flowchart

START
Read input / initialize variables
Process using conditions and loops
Display result / error message
STOP

6. Pseudocode

```
START
INPUT attended, total
IF total <= 0 OR attended < 0 OR attended > total THEN PRINT invalid input
ELSE     percentage = attended / total * 100     extra = 0     WHILE
(attended + extra) / (total + extra) * 100 < 75         extra = extra + 1
PRINT percentage and extra
STOP
```

5. Attendance Recovery Computation — Python Source Code

7. Python Source Code

```
attended = int(input("Enter classes attended: ")) total
= int(input("Enter total classes conducted: "))
required = 75

if total <= 0 or attended < 0 or attended > total:
    print("Invalid attendance details") else:
    percentage = attended / total * 100     print("Current
attendance:", round(percentage, 2), "%")

    extra = 0     while (attended + extra) / (total + extra) * 100
< required:         extra = extra + 1

    print("Additional classes to attend:", extra)
print("Total classes after recovery:", total + extra)
```

8. Test Cases

Test Input	Expected Result
60 attended, 80 total	Current attendance 75%; extra classes 0
50 attended, 80 total	Current attendance 62.5%; extra classes 40
85 attended, 80 total	Invalid attendance details

9. Sample Output Screenshot

SAMPLE CONSOLE OUTPUT

```
Enter classes attended: 50
Enter total classes conducted: 80
Current attendance: 62.5 %
Additional classes to attend: 40
Total classes after recovery: 120
```

10. Conclusion

The program uses a while loop to find the smallest number of additional attended classes required to reach a threshold.

6. ATM Transaction Sequence Simulation

Reference: Case Study 13

1. Problem Statement

Simulate basic ATM operations including deposits, withdrawals, balance enquiry, and transaction summary. **2.**

Objectives

• Maintain an account balance.

• Allow deposits and withdrawals through a menu.

• Reject invalid amounts and withdrawals beyond the balance or limit.

• Display transaction totals and final balance.

3. Assumptions

• This is a classroom simulation and does not access a real bank.

• Deposits and withdrawals must be positive.

• A withdrawal cannot exceed the balance or Rs. 10,000 per transaction.

4. Algorithm

1. Start and read the opening balance.
2. Repeat the menu until the user chooses Exit.
3. For deposit, add a valid amount to the balance.
4. For withdrawal, check amount, balance, and transaction limit.
5. For balance enquiry, display the balance.
6. At exit, display deposit total, withdrawal total, and final balance.
7. Stop.

5. Flowchart

START
Read input / initialize variables
Process using conditions and loops
Display result / error message
STOP

6. Pseudocode

```
START
INPUT balance
SET deposits = 0, withdrawals = 0
REPEAT
    DISPLAY menu
    INPUT choice
```

```
    IF choice = 1 THEN INPUT amount; IF amount > 0 ADD to balance and deposits
    ELSE IF choice = 2 THEN INPUT amount; validate amount, balance, and limit
    ELSE IF choice = 3 THEN PRINT balance
    ELSE IF choice = 4 THEN EXIT LOOP
    ELSE PRINT invalid choice
UNTIL exit
PRINT deposits, withdrawals, balance
STOP
```

6. ATM Transaction Sequence Simulation — Python Source Code

7. Python Source Code

```
balance = float(input("Enter opening balance: Rs. "))
deposits = 0 withdrawals = 0

if balance < 0:
    print("Invalid opening balance")
else:     while True:
        print("\n1. Deposit")
print("2. Withdraw")         print("3. Check
balance")         print("4. Exit")
choice = input("Enter your choice: ")

        if choice == "1":
            amount = float(input("Enter deposit amount: Rs. "))
if amount > 0:
                balance += amount
deposits += amount
print("Deposit successful")
else:                 print("Invalid
amount")

        elif choice == "2":
            amount = float(input("Enter withdrawal amount: Rs. "))
if amount <= 0:
                print("Invalid amount")
elif amount > balance:
                print("Insufficient balance")
elif amount > 10000:
                print("Withdrawal limit exceeded")
else:
                balance -= amount
withdrawals += amount
print("Withdrawal successful")

        elif choice == "3":
            print("Current balance: Rs.", round(balance, 2))
elif choice == "4":
            break         else:
print("Invalid choice")

    print("Total deposits: Rs.", round(deposits, 2))
print("Total withdrawals: Rs.", round(withdrawals, 2))
print("Final balance: Rs.", round(balance, 2))
```

8. Test Cases

Test Input	Expected Result
Opening 5000; deposit 1000; withdraw 2000; exit	Deposits 1000; withdrawals 2000; final balance Rs. 4000
Opening 500; withdraw 700	Insufficient balance
Opening 500; withdraw 12000	Withdrawal limit exceeded

9. Sample Output Screenshot

SAMPLE CONSOLE OUTPUT

```
Enter opening balance: Rs. 5000
1. Deposit
2. Withdraw
3. Check balance
4. Exit
Enter your choice: 1
Enter deposit amount: Rs. 1000
Deposit successful
Enter your choice: 2
Enter withdrawal amount: Rs. 2000
Withdrawal successful
Enter your choice: 4
Total deposits: Rs. 1000.0
Total withdrawals: Rs. 2000.0
Final balance: Rs. 4000.0
```

10. Conclusion

The program demonstrates menu-driven programming, loops, validation, and balance updates.

7. Student Timetable Conflict Detection

Reference: Case Study 23

1. Problem Statement

Represent class schedules using tuples and detect two classes that are scheduled on the same day and at the same time.

2. Objectives

• Store subject, day, and time in tuples.

• Compare each schedule with other schedules.

• Report any duplicate day-and-time slot.

• Display a message if no conflict exists.

3. Assumptions

• Each tuple contains a subject name, day, and single time-slot label.

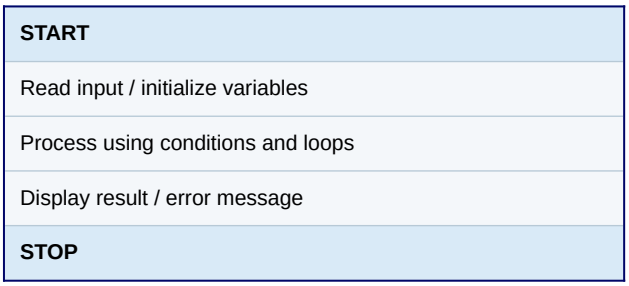
• Two classes conflict when both day and time are identical.

• The example checks exact slot matches, not partial overlaps.

4. Algorithm

1. Start.
2. Create a list of schedule tuples.
3. Set conflict flag to false.
4. Compare each pair of schedule entries using nested loops.
5. If day and time match, display the conflicting subjects and set flag to true.
6. If the flag remains false, display no conflicts.
7. Stop.

5. Flowchart



6. Pseudocode

```
START
SET schedule list of tuples
SET conflict = false
FOR i from 0 to length(schedule)-1
    FOR j from i+1 to length(schedule)-1
        UNPACK both tuples
        IF day1 = day2 AND time1 = time2 THEN
PRINT conflicting subjects
conflict = true IF conflict = false THEN PRINT
no conflicts
STOP
```

7. Student Timetable Conflict Detection — Python Source Code

7. Python Source Code

```
schedule = [     ("Python",
"Monday", "9 AM"),
    ("Maths", "Monday", "10 AM"),
    ("Physics", "Tuesday", "9 AM"),
    ("English", "Monday", "9 AM") ]

conflict_found = False
```

```
for i in range(len(schedule)):     for j in
range(i + 1, len(schedule)):
subject1, day1, time1 = schedule[i]
subject2, day2, time2 = schedule[j]

        if day1 == day2 and time1 == time2:
print("Timetable conflict:")
print(subject1, "and", subject2)
print("Day:", day1)             print("Time:",
time1)             conflict_found = True

if conflict_found == False:     print("No
timetable conflicts found")
```

8. Test Cases

Test Input	Expected Result
Example list shown in program	Python and English conflict on Monday at 9 AM
Change English to Tuesday, 10 AM	No timetable conflicts found
Two different days with same time	No conflict

9. Sample Output Screenshot

SAMPLE CONSOLE OUTPUT

```
Timetable conflict:
Python and English
Day: Monday
Time: 9 AM
```

10. Conclusion

The program demonstrates lists of tuples, tuple unpacking, nested loops, and comparison of schedule data.

8. Password Policy Validation Engine

Reference: Case Study 29

1. Problem Statement

Check a sample password against basic rules for length, uppercase letters, lowercase letters, digits, and special characters.

2. Objectives

• Read a password string.

• Check the minimum length.

• Check required character categories.

• Report failed conditions or accept the password.

3. Assumptions

• The password must contain at least eight characters.

• It must contain at least one uppercase letter, one lowercase letter, one digit, and one special character.

• This is a basic educational validator and does not check every rule mentioned in the case-study description.

4. Algorithm

1. Start.
2. Read the password.
3. Set valid to true.
4. Check length and each required character category.
5. Print a message for each failed condition and set valid to false.
6. Display valid or invalid based on the flag.

7. Stop.

5. Flowchart

START
Read input / initialize variables
Process using conditions and loops
Display result / error message
STOP

6. Pseudocode

```
START
INPUT password
SET valid = true
IF length(password) < 8 THEN PRINT message; valid = false
IF no uppercase character THEN PRINT message; valid = false
IF no lowercase character THEN PRINT message; valid = false
IF no digit THEN PRINT message; valid = false
IF no special character THEN PRINT message; valid = false
IF valid THEN PRINT password valid ELSE PRINT password invalid
STOP
```

8. Password Policy Validation Engine — Python Source Code

7. Python Source Code

```
password = input("Enter a password: ") valid
= True

if len(password) < 8:
    print("Password must have at least 8 characters")
valid = False

if not any(ch.isupper() for ch in password):
    print("Password must contain an uppercase letter")
valid = False

if not any(ch.islower() for ch in password):
    print("Password must contain a lowercase letter")
valid = False

if not any(ch.isdigit() for ch in password):
print("Password must contain a digit")     valid
= False

if not any(not ch.isalnum() for ch in password):
    print("Password must contain a special character")
valid = False

if valid:
    print("Password is valid")
else:     print("Password is
invalid")
```

8. Test Cases

Test Input	Expected Result
Hello@123	Password is valid
hello123	Missing uppercase and special character
Ab@1	Too short

9. Sample Output Screenshot

SAMPLE CONSOLE OUTPUT

```
Enter a password: Hello@123
Password is valid
```

10. Conclusion

The program uses string methods and conditional checks to validate a password against a simple policy.

9. Student Marks Matrix Analysis

Reference: Case Study 42

1. Problem Statement

Store marks for multiple students in a nested list, calculate each student's total and average, and identify the student with the highest total.

2. Objectives

• Accept marks for three subjects for each student.

• Store marks in a nested list.

• Calculate totals and averages.

• Identify the student with the highest total.

3. Assumptions

• There are three subjects, each out of 100.

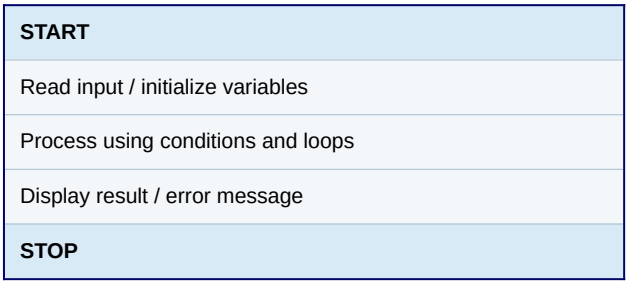
• The number of students must be positive.

• The example assumes marks entered are valid integers from 0 to 100.

4. Algorithm

1. Start.
2. Read the number of students and validate it.
3. For each student, read three subject marks and store them in a nested list.
4. Calculate each student's total and average.
5. Store each total in a separate list.
6. Find the maximum total and its student number.
7. Display each student's result and the topper.
8. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT n
IF n <= 0 THEN PRINT invalid number
ELSE
    SET marks = empty list
    FOR each student
        INPUT 3 subject marks
        ADD marks row to marks list     FOR
each student row         total = sum(row)
average = total / 3         STORE total
PRINT row, total, average     topper =
position of maximum total + 1     PRINT
topper and highest total
STOP
```

9. Student Marks Matrix Analysis — Python Source Code

7. Python Source Code

```
marks = [] n = int(input("Enter number of
students: "))

if n <= 0:
    print("Invalid number of students") else:
for i in range(n):         student_marks = []
print("Enter marks for student", i + 1)

        for j in range(3):
            mark = int(input("Enter marks for subject " + str(j + 1) + ": "))
student_marks.append(mark)

        marks.append(student_marks)

    totals = []
for i in range(n):
        total = sum(marks[i])         average =
total / 3         totals.append(total)
print("Student", i + 1, "marks:", marks[i])
print("Total:", total)         print("Average:",
round(average, 2))

    highest = max(totals)     topper =
totals.index(highest) + 1     print("Top
student: Student", topper)
print("Highest total:", highest)
```

8. Test Cases

Test Input	Expected Result
Student 1: 80,75,90; Student 2: 85,88,92	Student 1 total 245; Student 2 total 265; topper Student 2
One student: 60,60,60	Total 180; average 60; topper Student 1
n = 0	Invalid number of students

9. Sample Output Screenshot

SAMPLE CONSOLE OUTPUT

```
Student 1 marks: [80, 75, 90]
Total: 245
Average: 81.67
Student 2 marks: [85, 88, 92]
Total: 265
Average: 88.33
Top student: Student 2
Highest total: 265
```

10. Conclusion

The program demonstrates nested lists and loops, along with totals, averages, maximum values, and list indexing.

10. Food Waste Monitoring System

Reference: Case Study 53

1. Problem Statement

Track the number of meals prepared and served over several days, calculate leftover meals and wastage percentage, and warn when wastage exceeds an acceptable limit.

2. Objectives

• Accept the number of days and the acceptable wastage percentage.

• Read prepared and served meals for each day.

• Calculate leftover meals and wastage percentage.

• Flag days that exceed the limit.

3. Assumptions

• Prepared meals must be greater than zero.

• Served meals must be between zero and the number prepared.

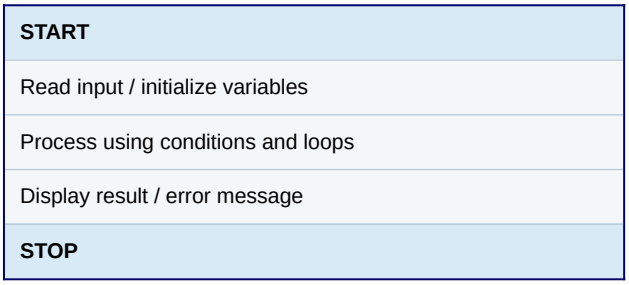
• The acceptable wastage percentage is between 0 and 100.

• A warning is displayed only when wastage is greater than the limit.

4. Algorithm

1. Start.
2. Read number of days and acceptable wastage limit.
3. Validate these values.
4. For each day, read prepared and served meals.
5. Validate meal counts.
6. Calculate leftovers and wastage percentage.
7. Compare wastage with the limit and display a warning if needed.
8. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT days, limit
IF days <= 0 OR limit < 0 OR limit > 100 THEN PRINT invalid input
ELSE
    FOR each day
        INPUT prepared, served
        IF prepared <= 0 OR served < 0 OR served > prepared THEN PRINT invalid meal details
ELSE             leftover = prepared - served             wastage = leftover / prepared * 100
PRINT leftover, wastage
            IF wastage > limit THEN PRINT warning
            ELSE PRINT within limit
STOP
```

10. Food Waste Monitoring System — Python Source Code

7. Python Source Code

```
days = int(input("Enter number of days: ")) limit =
float(input("Enter acceptable wastage percentage: "))

if days <= 0 or limit < 0 or limit > 100:
    print("Invalid input") else:     for i in
range(days):         print("Day", i + 1)
prepared = int(input("Enter meals prepared: "))
served = int(input("Enter meals served: "))

        if prepared <= 0 or served < 0 or served > prepared:
            print("Invalid meal details")
else:
            leftover = prepared - served             wastage =
leftover / prepared * 100             print("Meals leftover:",
leftover)             print("Wastage percentage:", round(wastage,
2), "%")

            if wastage > limit:
                print("Warning: Excessive food waste")
else:
                print("Food waste is within the limit")
```

8. Test Cases

Test Input	Expected Result
Prepared 200, served 170, limit 10	Leftover 30; wastage 15%; excessive waste warning
Prepared 100, served 95, limit 10	Leftover 5; wastage 5%; within limit
Prepared 0, served 0	Invalid meal details

9. Sample Output Screenshot

SAMPLE CONSOLE OUTPUT

```
Enter number of days: 1
Enter acceptable wastage percentage: 10
Day 1
Enter meals prepared: 200
Enter meals served: 170
Meals leftover: 30
Wastage percentage: 15.0 %
Warning: Excessive food waste
```

10. Conclusion

The program uses loops, input validation, arithmetic, and conditional statements to monitor food wastage.